

Ejercicios

Ejercicio 1. Clase Persona

Objetivo: Practicar constructores básicos y uso de this.

Crea una clase Persona con los siguientes atributos:

```
private String nombre;  
private int edad;
```

La clase debe tener:

1. Un constructor que reciba nombre y edad.
2. Un método mostrarDatos() que imprima el nombre y la edad.

Ejemplo de uso:

```
Persona p1 = new Persona("Ana", 25);  
p1.mostrarDatos();
```

Salida esperada

```
Nombre: Ana  
Edad: 25
```

Preguntas:

1. ¿Por qué usamos this.nombre = nombre?
2. ¿Qué ocurre si quitamos this en este caso?

Ejercicio 2. Sobrecarga de constructores

Objetivo: Reutilizar constructores de una misma clase mediante this().

Amplía la clase Persona para que tenga tres constructores:

```
Persona()  
Persona(String nombre)  
Persona(String nombre, int edad)
```

Reglas:

- Si no se indica nombre, debe usarse "Sin nombre".
- Si no se indica edad, debe usarse 0.
- Los constructores deben reutilizarse usando this().

Ejemplo de uso:

```
Persona p1 = new Persona();  
Persona p2 = new Persona("Carlos");  
Persona p3 = new Persona("Lucía", 30);  
  
p1.mostrarDatos();  
p2.mostrarDatos();  
p3.mostrarDatos();
```

Salida esperada

```
Nombre: Sin nombre  
Edad: 0  
  
Nombre: Carlos
```

Edad: 0

Nombre: Lucía

Edad: 30

Ejercicio 3. Clase CuentaBancaria

Objetivo: Combinar sobrecarga de constructores con validación de datos.

Crea una clase CuentaBancaria con:

```
private String titular;  
private double saldo;
```

Debe tener:

1. Un constructor que reciba solo el titular.
2. Un constructor que reciba titular y saldo inicial.
3. El constructor de un parámetro debe llamar al de dos parámetros usando this().
4. Si el saldo inicial es negativo, el saldo debe quedar en 0.

Ejemplo de creación de objetos:

```
CuentaBancaria c1 = new CuentaBancaria("Marta");  
CuentaBancaria c2 = new CuentaBancaria("Luis", 500);  
CuentaBancaria c3 = new CuentaBancaria("Eva", -100);
```

Métodos requeridos:

```
void mostrarCuenta()  
void ingresar(double cantidad)  
void retirar(double cantidad)
```

Reglas de negocio:

- No se puede ingresar una cantidad negativa.
- No se puede retirar una cantidad negativa.
- No se puede retirar más dinero del saldo disponible.

Ejercicio 4. Herencia básica con super

Objetivo: Inicializar atributos heredados mediante el constructor de la clase padre.

Crea una clase Animal con:

```
protected String nombre;  
protected int edad;
```

Debe tener un constructor y un método:

```
Animal(String nombre, int edad)  
void mostrarDatos()
```

Después crea una clase Perro que herede de Animal y añada:

```
private String raza;
```

El constructor de Perro debe ser:

```
Perro(String nombre, int edad, String raza)
```

Este constructor debe llamar al constructor de Animal usando super(nombre, edad).

Ejemplo de uso:

```
Perro p = new Perro("Toby", 4, "Labrador");  
p.mostrarDatos();
```

Salida esperada

```
Nombre: Toby  
Edad: 4  
Raza: Labrador
```

Ejercicio 5. Orden de ejecución de constructores

Objetivo: Razonar el orden de llamadas en una cadena de herencia.

Analiza el siguiente código sin ejecutarlo primero:

```
class A {  
    public A() {  
        System.out.println("Constructor de A");  
    }  
}  
  
class B extends A {  
    public B() {  
        System.out.println("Constructor de B");  
    }  
}  
  
class C extends B {  
    public C() {  
        System.out.println("Constructor de C");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        C objeto = new C();  
    }  
}
```

Preguntas:

1. ¿Qué se imprime por pantalla?
2. ¿Por qué se ejecuta primero el constructor de A?
3. ¿Hay una llamada a super() aunque no aparezca escrita?
4. Después de responder, ejecuta el programa y comprueba tu predicción.

Ejercicio 6. Clase Vehiculo y clase Coche

Objetivo: Aplicar herencia y sobrescritura de métodos de presentación.

Crea una clase Vehiculo con:

```
private String marca;  
private String modelo;
```

Debe tener:

```
Vehiculo(String marca, String modelo)  
void mostrarDatos()
```

Después crea una clase Coche que herede de Vehiculo y añada:

```
private int numeroPuertas;
```

Debe tener el constructor:

```
Coche(String marca, String modelo, int numeroPuertas)
```

Usa super() para inicializar los atributos heredados.

Ejemplo de uso:

```
Coche c = new Coche("Toyota", "Corolla", 5);  
c.mostrarDatos();
```

Salida esperada

```
Marca: Toyota  
Modelo: Corolla  
Número de puertas: 5
```

Ejercicio 7. Reutilización de constructores con this()

Objetivo: Centralizar la inicialización de atributos en el constructor más completo.

Crea una clase Producto con:

```
private String nombre;  
private double precio;  
private int stock;
```

Debe tener los siguientes constructores:

```
Producto(String nombre)  
Producto(String nombre, double precio)  
Producto(String nombre, double precio, int stock)
```

Reglas:

- Si solo se indica el nombre, el precio será 0 y el stock será 0.
- Si se indica nombre y precio, el stock será 0.
- Todos los constructores deben acabar llamando al constructor más completo.
- No se permite precio negativo.
- No se permite stock negativo.

Ejemplo de creación de objetos:

```
Producto p1 = new Producto("Ratón");  
Producto p2 = new Producto("Teclado", 25.99);  
Producto p3 = new Producto("Monitor", 199.99, 10);
```

Ejercicio 8. Herencia con varios constructores

Objetivo: Combinar this() y super() en clases relacionadas por herencia.

Crea una clase Empleado con:

```
private String nombre;  
private double salario;
```

Constructores:

```
Empleado(String nombre)
Empleado(String nombre, double salario)
```

Reglas de Empleado:

- Si no se indica salario, será 1000.
- Usa this() para reutilizar constructores.

Después crea una clase Programador que herede de Empleado y añada:

```
private String lenguajePrincipal;
```

Constructores de Programador:

```
Programador(String nombre)
Programador(String nombre, double salario, String lenguajePrincipal)
```

Reglas de Programador:

- Si no se indica lenguaje, será "Java".
- El constructor simple de Programador debe reutilizar el constructor completo usando this().
- El constructor completo debe llamar a super(nombre, salario).

Ejemplo de creación de objetos:

```
Programador p1 = new Programador("Laura");
Programador p2 = new Programador("David", 2200, "Python");
```

Ejercicio 9. Detectar errores

Objetivo: Identificar fallos típicos relacionados con this y super.

Indica qué errores hay en el siguiente código y corrígelo:

```
class Persona {
    private String nombre;

    public Persona(String nombre) {
        nombre = nombre;
    }
}

class Alumno extends Persona {
    private String curso;

    public Alumno(String nombre, String curso) {
        this.curso = curso;
    }
}
```

Preguntas:

1. ¿Qué problema hay en nombre = nombre?
2. ¿Por qué falla el constructor de Alumno?
3. ¿Dónde debería ir la llamada a super(nombre)?
4. Escribe una versión corregida del código.

Ejercicio 10. Mini proyecto final: Biblioteca

Objetivo: Integrar constructores, herencia, super() y sobrescritura de métodos.

Crea un pequeño sistema de gestión de biblioteca con las siguientes clases.

Clase MaterialBiblioteca

Atributos:

```
private String titulo;  
private String autor;  
private int anioPublicacion;
```

Constructor y método:

```
MaterialBiblioteca(String titulo, String autor, int anioPublicacion)  
void mostrarInfo()
```

Clase Libro

Hereda de MaterialBiblioteca y añade:

```
private int numeroPaginas;
```

Constructor:

```
Libro(String titulo, String autor, int anioPublicacion, int numeroPaginas)
```

Clase Revista

Hereda de MaterialBiblioteca y añade:

```
private int numeroEdicion;
```

Constructor:

```
Revista(String titulo, String autor, int anioPublicacion, int numeroEdicion)
```

Clase DVD

Hereda de MaterialBiblioteca y añade:

```
private int duracionMinutos;
```

Constructor:

```
DVD(String titulo, String autor, int anioPublicacion, int duracionMinutos)
```

Requisitos:

1. Todas las subclases deben llamar al constructor de MaterialBiblioteca usando super().
2. Cada clase debe sobrescribir mostrarInfo() para mostrar también sus atributos propios.
3. Crea una clase Main donde se creen varios objetos.
4. Muestra la información de todos los objetos por pantalla.

Objetos mínimos a crear en Main:

```
Libro libro = new Libro("El Quijote", "Miguel de Cervantes", 1605, 863);  
Revista revista = new Revista("National Geographic", "Varios autores", 2023, 120);  
DVD dvd = new DVD("Matrix", "Lana y Lilly Wachowski", 1999, 136);
```

Preguntas de repaso

1. ¿Qué diferencia hay entre this y super?
2. ¿Qué diferencia hay entre this() y super()?
3. ¿Puede un constructor llamar a otro constructor de la misma clase?
4. ¿Puede aparecer super() en cualquier línea del constructor?
5. ¿Qué ocurre si una clase hija no llama explícitamente a super()?
6. ¿Qué ocurre si la clase padre no tiene constructor vacío?
7. ¿Se heredan los constructores?
8. ¿Puede un constructor devolver un valor?
9. ¿Por qué no tiene sentido escribir void Persona() como constructor?
10. ¿En qué orden se ejecutan los constructores cuando hay herencia?

Ejercicio extra tipo examen

Dado el siguiente código, responde sin ejecutarlo primero:

```
class Vehiculo {
    public Vehiculo() {
        System.out.println("Vehículo creado");
    }

    public Vehiculo(String marca) {
        System.out.println("Vehículo de marca " + marca);
    }
}

class Moto extends Vehiculo {
    public Moto() {
        this("Yamaha");
        System.out.println("Moto creada");
    }

    public Moto(String marca) {
        super(marca);
        System.out.println("Moto de marca " + marca);
    }
}

public class Main {
    public static void main(String[] args) {
        Moto m = new Moto();
    }
}
```

Preguntas:

1. ¿Qué constructor se ejecuta primero?
2. ¿Qué imprime el programa?
3. ¿Por qué this("Yamaha") debe estar en la primera línea del constructor?
4. ¿Por qué super(marca) debe estar en la primera línea del otro constructor?
5. Ejecuta el código después de responder y compara el resultado con tu predicción.

Criterios de entrega

- El código debe compilar sin errores.

- Cada clase debe estar en su propio archivo si se trabaja con un proyecto Java estándar.
- Los nombres de clases, atributos y métodos deben respetar los enunciados.
- Los constructores deben usar `this()` y `super()` cuando el ejercicio lo pida explícitamente.
- El alumno debe poder explicar el orden de ejecución de los constructores en los ejercicios de herencia.